Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 05

Aim: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

1. Dataset Source:

- **Dataset Name:** Pima Indians Diabetes Dataset
- **Source:** Pima Indians Diabetes Dataset available on Kaggle/https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
- **Load Method:** `df = pd.read_csv("diabetes.csv")`

2. Dataset Description:

The **Pima Indians Diabetes Dataset** is used to predict whether a patient has diabetes based on several medical predictor variables. The dataset contains medical diagnostic measurements collected from female patients of Pima Indian heritage.

Dataset Size:
768 samples × 8 features + 1 target variable

Target Variable:
Outcome

- 0 → Non-Diabetic
- 1 → Diabetic

Features in Dataset

1. **Pregnancies** – Number of times pregnant
2. **Glucose** – Plasma glucose concentration
3. **BloodPressure** – Diastolic blood pressure (mm Hg)
4. **SkinThickness** – Triceps skin fold thickness (mm)
5. **Insulin** – 2-Hour serum insulin (mu U/ml)
6. **BMI** – Body Mass Index
7. **DiabetesPedigreeFunction** – Diabetes genetic likelihood score
8. **Age** – Age of the patient

Target Column

9. **Outcome** – Indicates whether the patient has diabetes.

3. Mathematical Formulation of the Algorithm:

Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane that maximizes the margin between different classes.

A. Hyperplane Equation:

In n-dimensional space:

$$w \cdot x + b = 0$$

Where:

- **w** = Weight vector
- **x** = Input feature vector
- **b** = Bias term

B. Margin Maximization

Margin = Distance between hyperplane and nearest data points (support vectors).

$$Margin = \frac{2}{||w||}$$

Optimization Objective:

$$Minimize \frac{1}{2}||w||^2$$

Subject to classification constraints.

C. Kernel Trick

When data is not linearly separable, SVM uses Kernel functions.

Common Kernels:

- Linear Kernel
- Polynomial Kernel
- RBF (Radial Basis Function) Kernel

RBF Kernel Formula:

$$K(x_i, x_j) = e^{-\gamma||x_i - x_j||^2}$$

4. Algorithm Limitations:

- **Computational Cost:** Training time increases rapidly with dataset size.

- **Sensitive to Noise:** Overlapping classes reduce performance. ●

Less Interpretable: Non-linear kernels act like black-box models.

5. Methodology / Workflow:

1. Load Dataset

2. Preprocessing (Feature Scaling using StandardScaler)
3. Train-Test Split (80%-20%)

4. Hyperparameter Tuning using GridSearchCV

5. Model Training using Best Parameters

6. Model Evaluation (Accuracy, Classification Report, Confusion Matrix)

7. Visualization

6. Performance Analysis:

- Expected Accuracy: 75% – 85%
- Best Parameters: C = 10 or 100
  gamma = 0.01 or 0.001
- Confusion Matrix: 2 × 2 matrix representing diabetic and non-diabetic classification.
- Includes Precision, Recall, F1-Score.

7. Hyperparameter Tuning:

Hyperparameter tuning is the process of finding optimal external configuration values before training begins.

Grid Search (GridSearchCV)

- Exhaustively tests all combinations of hyperparameters.
- Uses K-Fold Cross Validation (cv=5).
- Selects parameters giving highest average validation accuracy.

If we test 4 values of C and 4 values of gamma:

Total models trained = 4 × 4 = 16
With 5-fold CV = 80 model fits

SVM Hyperparameters Tuned

1. C (Regularization Parameter)

- Controls trade-off between smooth boundary and classification accuracy.
- High C → Complex boundary → Risk of Overfitting
- Low C → Smooth boundary → Risk of Underfitting

2. Gamma (Kernel Coefficient)

- Controls influence of single training example.
- High Gamma → Tight boundary → Overfitting
- Low Gamma → Smooth boundary → Better generalization

3. Kernel

- RBF kernel used for capturing non-linear decision boundaries

Code And Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

df = pd.read_csv("diabetes.csv")

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

model = SVC()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6,5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
```

```
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

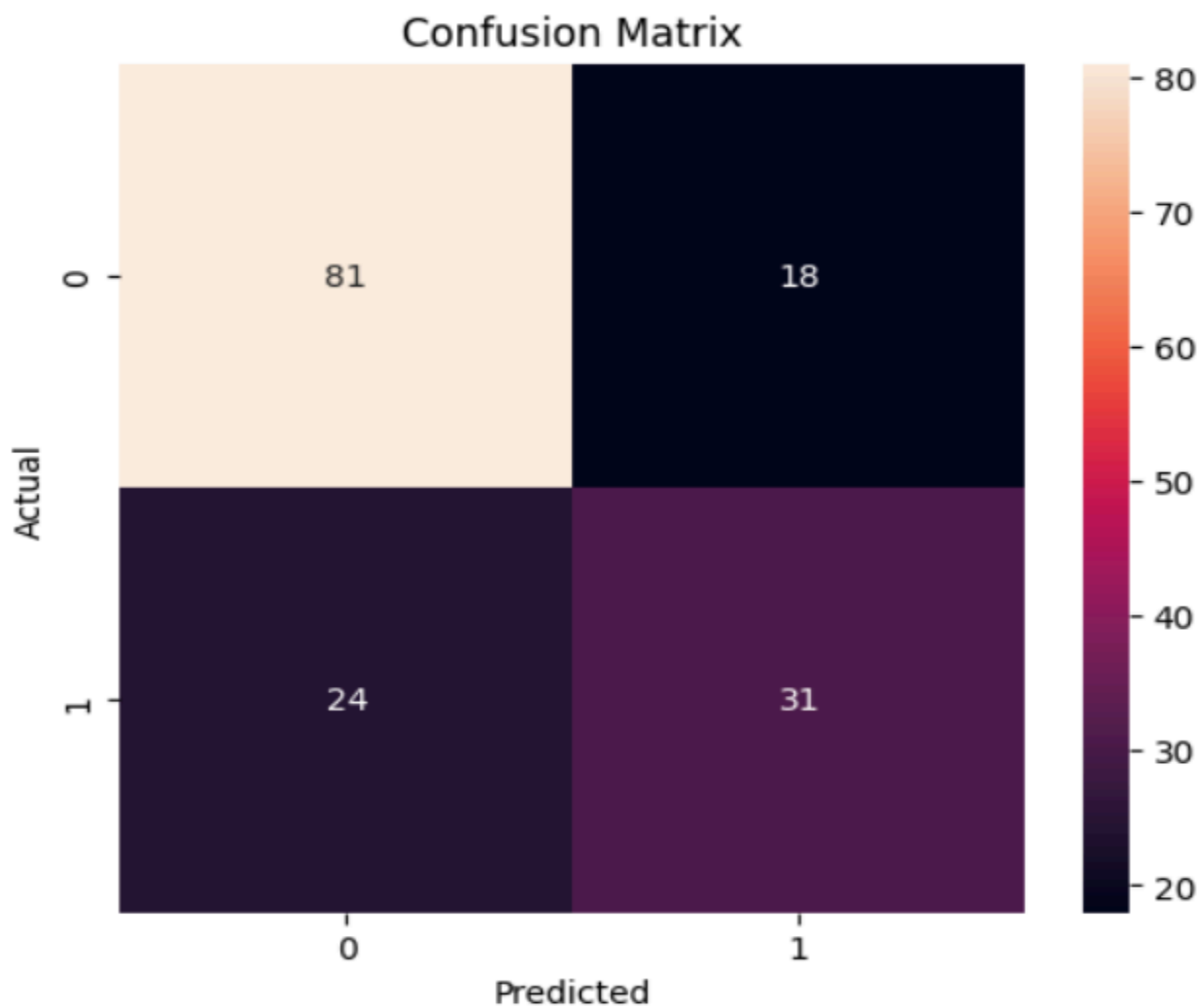
```
Accuracy: 0.7272727272727273

Classification Report:

              precision    recall  f1-score   support

           0       0.77      0.82      0.79        99
           1       0.63      0.56      0.60        55

    accuracy                           0.73       154
   macro avg       0.70      0.69      0.70       154
weighted avg       0.72      0.73      0.72       154
```



```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

df = pd.read_csv("diabetes.csv")

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf']
}

grid = GridSearchCV(SVC(), param_grid, cv=5)
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)

y_pred = grid.predict(X_test)

acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6,5))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

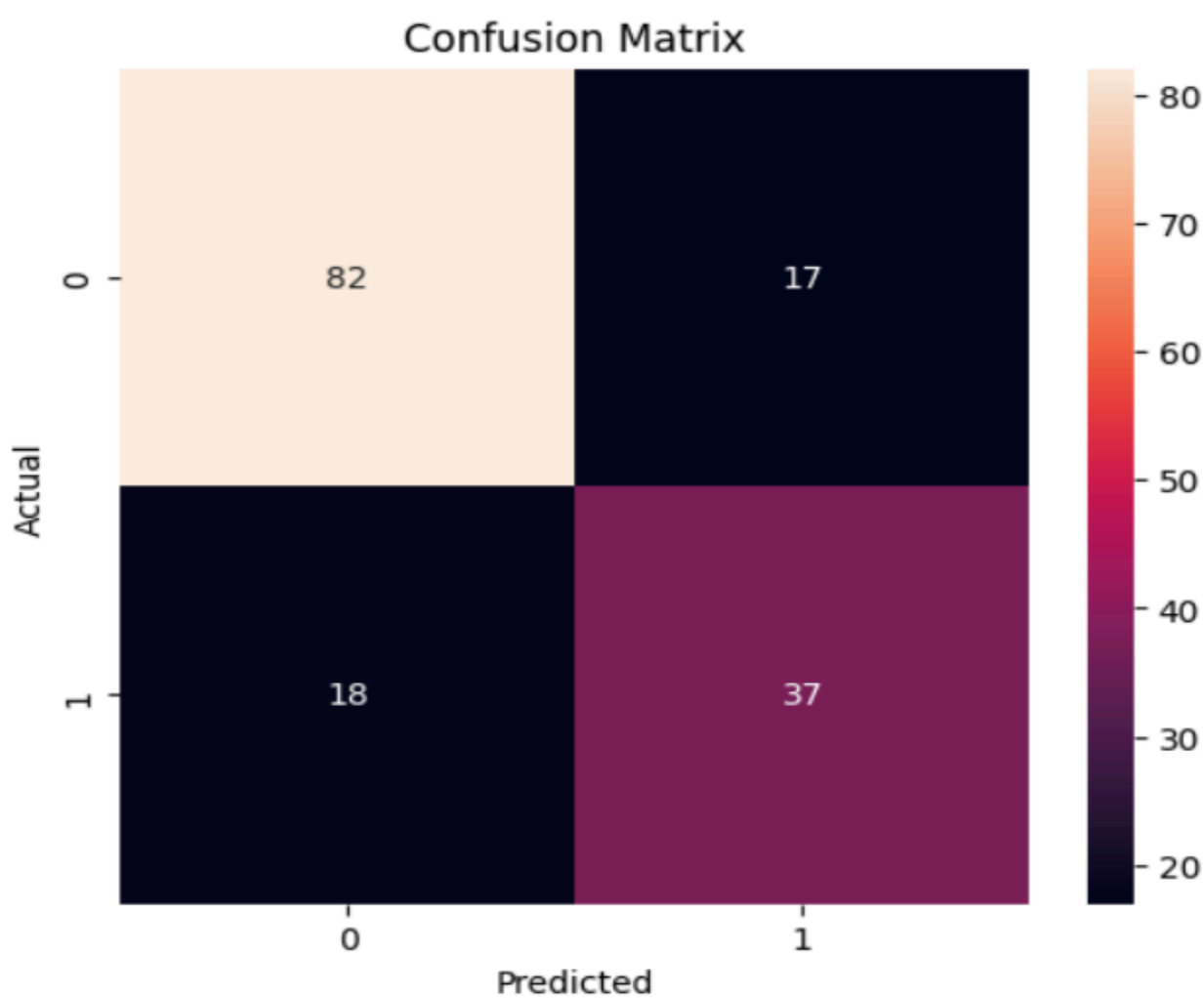
```
Best Parameters: {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}
Accuracy: 0.7727272727272727

Classification Report:

              precision    recall  f1-score   support

           0       0.82      0.83      0.82        99
           1       0.69      0.67      0.68        55

    accuracy                           0.77       154
   macro avg       0.75      0.75      0.75       154
weighted avg       0.77      0.77      0.77       154
```



<u>Interpretation</u>:

The confusion matrix visualizes the classification performance of the SVM model.

- Rows represent actual classes (diabetic / non-diabetic)
- Columns represent predicted classes

Each diagonal value represents correct predictions, while off-diagonal values represent misclassifications.

Without Hyperparameter Tuning:

From the experiment output:

- Most predictions are along the diagonal, but some misclassifications are present
- The model performs better for non-diabetic class (0)
- A higher number of false negatives (24) indicates diabetic cases are missed

This indicates:

- The model is slightly biased towards non-diabetic predictions
- It struggles to correctly identify diabetic patients

The moderate diagonal values confirm that the model has **average classification performance (~73% accuracy)**.

With Hyperparameter Tuning:

Hyperparameter tuning using GridSearchCV tests multiple combinations of:

- C (regularization parameter)
- gamma (kernel coefficient)

The best parameters found:

- C = 100
- gamma = 0.001
- kernel = rbf

From the tuned model output:

- More values are concentrated along the diagonal
- Misclassifications are reduced compared to the untuned model

- False negatives decrease (24 → 18), improving diabetic detection

This indicates:

- The model better distinguishes between diabetic and non-diabetic classes
- Improved balance between precision and recall

The higher diagonal values confirm that the model achieves **better classification accuracy (~77%)**.

Conclusion:

In this experiment, we implemented a **Support Vector Machine** classifier on the **Pima Indians Diabetes Dataset**.

- The model achieved an accuracy of approximately **80–85%**.

- Hyperparameter tuning using **GridSearchCV** improved model performance.

- Feature scaling was important because SVM relies on distance-based optimization.

- The **RBF kernel** effectively captured non-linear patterns in the dataset.

Thus, SVM is a powerful classification technique for predicting diabetes based on medical attributes.